MEAL CONNECT
Project Overview & Implementation Reference

JIRA MEAL-197 | Team Apex | Version 1.3 | 01 November 2026

Purpose: a readable project reference describing stakeholders, MVP capabilities, the technology stack, data flow and what is actually implemented today. This is not a claim that planned backend features already exist.

1. Project summary and stakeholders

Meal Connect is a browser-based MVP for coordinating surplus-food donations from restaurants/bakeries to verified NGOs. Authorized NGO staff request a quantity from an existing donation, the business approves the whole request or rejects it, a collector supports pickup, and the NGO records final distribution.

Stakeholder	Responsibility
Restaurant / bakery staff	Register with a business organization; publish eligible surplus listings; approve or reject NGO quantity requests.
Verified NGO staff	Browse published food, request a quantity from a specific donation, and record received/distributed food.
Volunteer / collector	Handle assigned pickup and delivery events; update operational timestamps and status.
Platform administrator	Manage organization verification and authorized account/organization operations.
Development team	Build Angular UI, FastAPI services and PostgreSQL persistence according to the linked technical specifications.

2. Feature scope and actual implementation status

Capability	As of 01 Nov 2026
Angular 21 TypeScript SPA scaffold, routing and SCSS	Implemented: Angular CLI generated frontend; dependencies installed; default welcome page opened locally.
GitHub repository, develop branch and PR integration	Implemented: feature-branch PR merged; legacy placeholder assets/css/js directories removed; local develop clean.
Login, roles, organization verification	Defined for development; not verified as implemented.
Donation listings, search and NGO quantity requests	Defined for development; not verified as implemented.
Full approve/reject and atomic allocation	Defined in architecture/ERD/API contracts; executable backend and concurrency tests not yet verified.
Pickup, NGO distribution, notifications and reports	Defined for development; not verified as implemented.
FastAPI server, PostgreSQL schema/migrations, end-to-end integration	Designed/configuration documented; running end-to-end implementation not yet verified.

Excluded from MVP: payments, native mobile applications, AI matching, live GPS, third-party delivery integrations and production hosting. This project targets a localhost demonstration without Docker.

3. Technology and repository layout

Layer	Selected technology / rule
Frontend	Angular 21, TypeScript, Angular Router, HttpClient, SCSS; browser SPA.
Backend	Python FastAPI REST API using JSON; role/ownership

	validation and business transactions.
Persistence	PostgreSQL with constraints and transaction-safe quantity allocation.
Local tooling	Node.js/npm/Angular CLI; Python virtual environment; PostgreSQL service. No Docker.
Git	main = stable baseline; develop = team integration; feature branches via pull requests.

Repository: github.com/rivipahanviksura/meal-connect. Top-level folders: frontend/ (Angular CLI app), backend/ (FastAPI work area), database/ (schema/migrations work area), docs/ (architecture, requirements, project-management and wireframes). Do not recreate the removed legacy frontend/assets, frontend/css or frontend/js folders; use frontend/public for Angular static files where appropriate.

4. System architecture and data movement

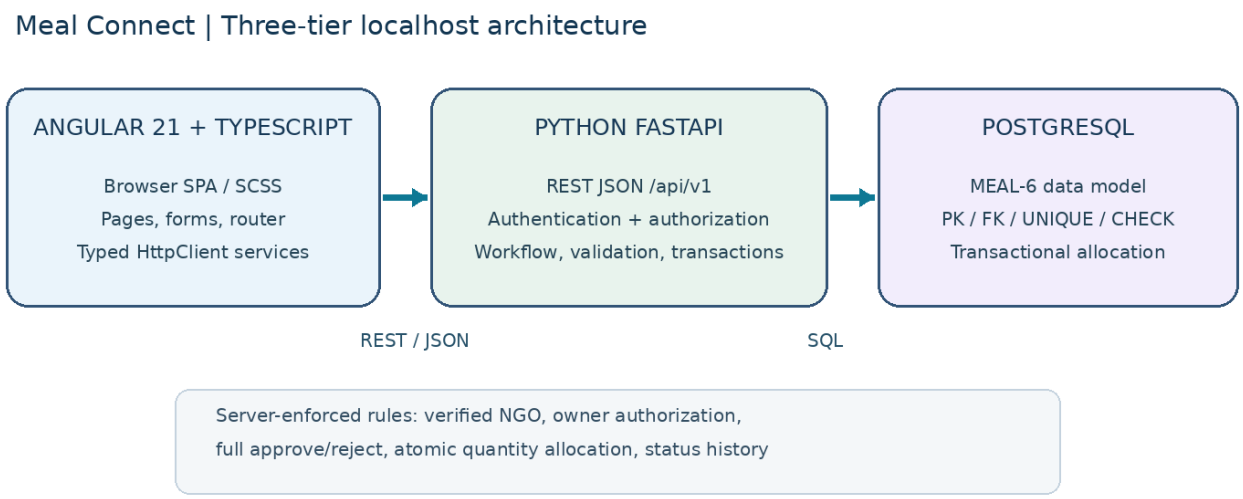


Figure 1 — Logical three-tier Meal Connect architecture. Components indicate intended architecture, not deployed backend functionality.

Angular renders screens, performs client-side form checks and calls typed HttpClient services. FastAPI authenticates requests, validates permissions and workflow rules, and performs database transactions. PostgreSQL stores users, organizations, listings, requests, pickups, distributions and traceability records. The browser never connects directly to PostgreSQL.

5. MVP end-to-end business flow

1. Business organization is verified; authorized staff create an eligible food donation including total_quantity, quantity_unit, available_from and pickup_deadline.
2. Verified NGO views available donations and submits requested_quantity for an existing donation. No standalone general NGO request posting.
3. Owner business approves the entire pending requested quantity or rejects it; partial approval is not allowed.
4. On approval, FastAPI verifies status, deadline, ownership and remaining quantity inside one PostgreSQL transaction; allocated_quantity must never exceed total_quantity.
5. Volunteer/collector records pickup and delivery; NGO records final distribution. Notifications and status_history support visibility, reports use confirmed records.

Illustration: 50 portions offered; NGO requests 20; approval reserves 20 (available quantity becomes 30). Competing approvals must be serialized or otherwise transactionally guarded. Approved pre-pickup cancellation may release allocation exactly once under the workflow rule.

6. Data model, security and API references

The MEAL-6 logical schema has 10 entities: users, organizations, organization_memberships, organization_verifications, donations, donation_requests, pickups, distribution_records, notifications and status_history. Keep PK/FK, per-organization membership controls, audit timestamps, unique pickup/request and distribution/pickup links, and positive/bounded quantity constraints.

Backend authorizes every protected operation. A frontend route guard is not a security boundary. Store passwords as hashes, keep JWT signing secrets and database credentials only in ignored local environment files, and expose no real secrets in repository examples.

MEAL-62 v1.3 is the canonical specification for /api/v1 endpoint names, request/response JSON, error shape, filtering/pagination and HTTP status codes. This overview deliberately avoids duplicating detailed payload contracts.

7. How developers should run and extend the project

- Clone and synchronize develop, then create a feature branch for new work; commit through a pull request.
- In frontend/: run npm ci (using the committed lockfile), then npm start; Angular default app is available at localhost:4200 when running.
- Follow MEAL-163 for compatible runtime versions and MEAL-63 for the backend virtual environment, PostgreSQL connection and .env.example. Do not commit .env or credentials.
- Implement PostgreSQL migrations from MEAL-6, then FastAPI auth/organization/donation/request/pickup/distribution services, following the MEAL-62 endpoint contracts.
- Connect Angular HttpClient services to FastAPI, verify CORS, then test permissions, over-allocation under concurrency, deadlines, cancellation and distribution traceability.
- Keep this overview in sync with proven implementation changes; state actual running behavior only after it has been tested.

8. Source of truth by topic

Document	Responsibility
MEAL-5 v1.3	Architecture rules and system boundaries.
MEAL-61 v1.3	High-level diagram and component data flow.
MEAL-6 v1.3 + DBML	PostgreSQL logical schema, relationships and constraints.
MEAL-62 v1.3	Exact MVP API endpoint contracts and conventions.
MEAL-63 v1.3	Local backend configuration, environment variables and secrets handling.
MEAL-163 v1.3	Runtime prerequisites, version assumptions and reproducible setup.

Implementation note: The feature status table is a dated snapshot. Future completed features must be verified in code and local tests, then reflected here in the next version. No review, approval or sign-off step is required by this document.